AI-DRIVEN CODE QUALITY SPRING BOOT ASSURANCE ENGINE AS PLUGIN

25-26J-451

Project Proposal Report

Jameela M.J.F

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering
Sri Lanka Institute of Information Technology Sri Lanka

August 2025

AI-DRIVEN CODE QUALITY SPRING BOOT ASSURANCE ENGINE AS PLUGIN

25-26J-451

Project Proposal Report

B.Sc. (Hons) Degree in Information Technology Specialized in Software Engineering

Department of Computer Science and Software Engineering

Sri Lanka Institute of Information Technology Sri Lanka

August 2025

DECLARATION

Declaration

We declare that this is our own work, and this proposal does not incorporate without acknowledgement any material previously submitted for a degree or diploma in any other university or institute of higher learning and to the best of our knowledge and belief it does not contain any material previously published or written by another person except where the acknowledgement is made in the text.

Name	Student ID	Signature
Janeela MJF	IT22060662	Jameela

Signature of the Supervisor: Relini
Date: 28/08/2025

Signature of the Co-Supervisor: S.S.Kumarasinghe 27/08/25
Date:

ABSTRACT

The rapid adoption of Spring Boot for enterprise Java applications has highlighted significant challenges in maintaining code quality, including inconsistent adherence to architectural patterns, manual detection of anti-patterns, and lack of personalized feedback. This proposal outlines the development of an AI-Driven Code Quality Spring Boot Assurance Engine as an IDE component, integrated into environments like IntelliJ IDEA, to address these issues. Spring Boot's widespread use in building microservices and web applications necessitates advanced tools capable of understanding complex architectural styles like Layered, Hexagonal, and Clean Architecture. Existing static analysis tools, such as SonarQube, Qodana, and CodeScene, rely on generic metrics and rules that often miss Spring Boot-specific anti-patterns, including embedding business logic in Controllers or inefficient transaction handling. They also lack adaptability to project-specific coding conventions, resulting in irrelevant recommendations and diminished developer productivity. Our proposed engine leverages a fine-tuned Code Llama model to perform architecture-aware, real-time analysis of Spring Boot projects. It automatically recognizes core layers—Controller, Service, Repository, and Entity—and computes dynamic quality metrics, including cohesion, query efficiency, and transaction correctness. The engine identifies anti-patterns and generates contextual, explainable recommendations with actionable fixes grounded in Spring Boot best practices. An adaptive learning module allows continuous refinement of suggestions through developer feedback, ensuring alignment with team-specific standards and improving maintainability and scalability. Evaluation will involve user testing on real-world projects to measure issue detection accuracy, metric relevance, and user acceptance.This approach promises to enhance software reliability by providing precise, real-time, architecture-sensitive feedback, reducing debugging time, and promoting best practices tailored for Spring Boot development.

Keywords: software quality assurance, Spring Boot, Code Llama, architecture-aware analysis, real-time code analysis, adaptive learning, IDE integration, quality metrics

Table of Contents

DECLARATION i
ABSTRACT ii
LIST OF FIGURES iv
LIST OF TABLES v
LIST OF ABBREVIATIONS vi
1. INTRODUCTION 1
1.1. Background 1
1.2. Literature Review 2
1.3. Research Gap 5
1.4. Research Problem 7
2. OBJECTIVES 9
2.1. Main Objectives 9
2.2. Specific Objectives 10
3. METHODOLOGY 12
3.1. System Architecture 12
3.2. Software Solution 13
3.3. Commercialization 14
3.4. Future Scope 15
4. PROJECT REQUIREMENTS 16
4.1. Functional Requirements 16
4.2. Non-Functional Requirements 18
4.3. System Requirements 19
4.4. Work breakdown structure 20
4.5. Gantt Chart 20
5. BUDGET JUSTIFICATION 21
6. REFERENCES 22

LIST OF FIGURES

Figure 1: Comparison of Spring Boot-Specific Issue Detection Accuracy 2
Figure 2: System Architecture Diagram ..... **Error! Bookmark not defined.**
Figure 3 : Work breakdown structure..... 20

LIST OF TABLES

Table 1: Comparison of Proposed Engine with Existing Tools and Systems 9
Table 2 : Gantt Chart..... 20

LIST OF ABBREVIATIONS

Abbreviation	Description
AI	Artificial Intelligence
AST	Abstract Syntax Tree
CC	Cyclomatic Complexity
CPU	Central Processing Unit
GDPR	General Data Protection Regulation
HTTP	Hypertext Transfer Protocol
IC	Interaction Complexity
IDE	Integrated Development Environment
JPA	Java Persistence API
JSON	JavaScript Object Notation
LKR	Sri Lankan Rupee
LLM	Large Language Model
LOC	Lines of Code
ML	Machine Learning
NOM	Number of Methods per Class
QA	Quality Assurance
RAM	Random Access Memory
RCI	Relative Complexity Index
SQA	Software Quality Assurance
SVM	Support Vector Machine
USD	United States Dollar
VS	Visual Studio (Code)
YAML	YAML Ain't Markup Language

1. INTRODUCTION

1.1. Background

Spring Boot has revolutionized enterprise application development by offering a streamlined, convention-over-configuration framework for building microservices and web applications. Its layered architecture—comprising Controllers for HTTP request handling, Services for business logic, Repositories for data access, and Entities for data modeling—enables rapid development but introduces significant challenges in maintaining code quality [1]. Common issues include anti-patterns such as business logic leaking into Controllers, improper use of annotations (e.g., @Controller vs. @Service), and inefficient transaction management, which compromise maintainability, scalability, and reliability [2]. These challenges are exacerbated by the complexity of adhering to architectural patterns like Layered, Hexagonal, or Clean Architecture, often leading to increased debugging time and reduced developer productivity [3].

Traditional static code analysis tools, such as SonarQube, Qodana, and Checkstyle, rely on generic metrics like Lines of Code (LOC) and Cyclomatic Complexity (CC), which lack the contextual awareness needed to address Spring Boot-specific best practices [1] , [4]. For instance, studies highlight that rule-based tools often fail to detect framework-specific anti-patterns, resulting in irrelevant recommendations that do not align with project conventions [4]. Recent advancements in machine learning (ML) offer a promising solution for software quality assurance (SQA) by enabling adaptive, real-time analysis [5]. Research on code smells categorizes issues like bloaters and coupling problems, proposing metrics such as Relative Complexity Index (RCI) and Interaction Complexity (IC) to enhance detection, though these are not IDE-integrated or Spring Boot-specific [1]. ML-based static analysis has shown potential for identifying hidden errors, but its application to Spring Boot remains limited [3]. Similarly, defect prediction models using Random Forest and SVM focus on prediction rather than real-time quality assurance [6]. Quality assurance analytics dashboards provide insights into QA practices but lack framework-specific focus [7]. Recent AI advancements in Spring Boot, such as automated test generation, suggest potential for quality assurance applications [8] , [9].

This project proposes an AI-driven code quality assurance engine integrated as a component within popular IDEs (e.g., IntelliJ IDEA, Eclipse, VS Code) to address these challenges. The engine leverages a fine-tuned Code Llama model to perform real-time, architecture-aware

analysis, explicitly configured with the project's architectural pattern (e.g., Layered, Hexagonal, Clean) via a setup wizard. It identifies Spring Boot layers, computes dynamic quality metrics—such as REST endpoint completeness for Controllers, business logic cohesion for Services, query efficiency for Repositories, and data model integrity for Entities—and detects anti-patterns like misplaced logic or tight coupling. The engine generates contextual, explainable recommendations with actionable fixes, referencing Spring Boot best practices, and adapts to team-specific conventions through developer feedback, enhancing productivity and code reliability [10]. Unlike tools like SonarQube or CodeScene, which lack architecture-specific metrics and adaptive learning, this engine offers precise, framework-tailored feedback directly in the IDE, reducing context-switching and debugging time. Figure 1. illustrates the comparative accuracy of issue detection in Spring Boot applications across traditional tools and the proposed engine, demonstrating its potential to outperform existing solutions.

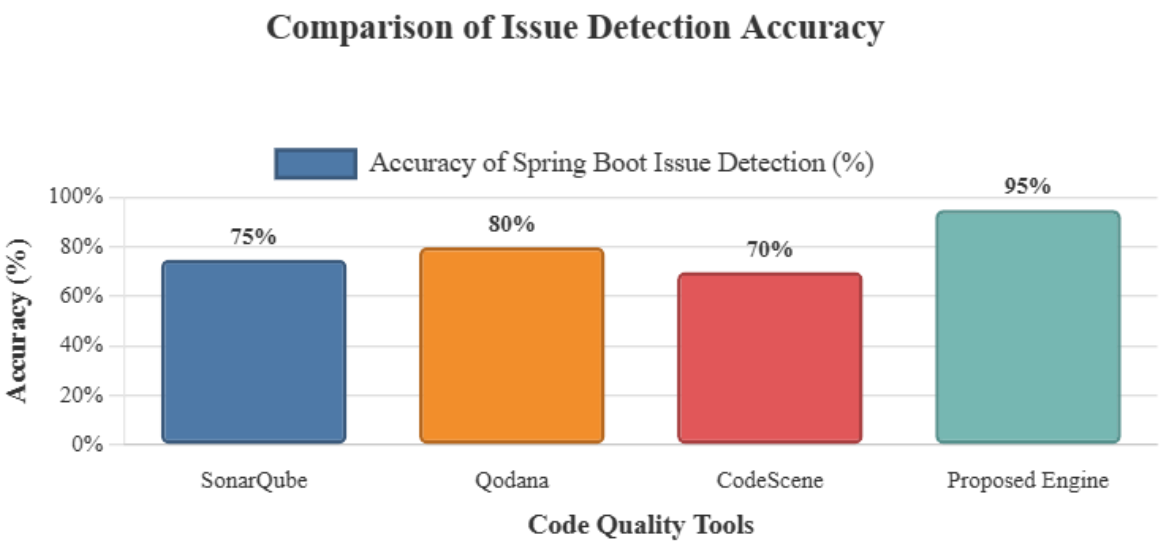


Figure 1: Comparison of Spring Boot-Specific Issue Detection Accuracy

1.2. Literature Review

The literature on software quality assurance (SQA) highlights the critical need for advanced tools to ensure code quality in modern software development, particularly for frameworks like Spring Boot, which is widely adopted for building enterprise-grade microservices and web applications.

Kataieva and Nemec (2025) emphasize the role of code analysis metrics in evaluating software quality, identifying code smells—such as bloaters, dispensable elements, and coupling issues—that compromise maintainability and reliability [1]. They propose metrics like Lines of Code (LOC), Cyclomatic Complexity (CC), Number of Methods per Class (NOM), and novel indicators like Relative Complexity Index (RCI) and Interaction Complexity (IC), implemented via a Maven plugin for Spring Boot projects. While their metrics enhance detection of structural issues, the approach lacks real-time IDE integration and adaptability to project-specific conventions, limiting its utility in dynamic development workflows [1].

Sultanow et al. (2025) explore machine learning (ML)-based static code analysis for large-scale systems, using association rule mining to uncover hidden errors in mission-critical applications [3]. Their prototype demonstrates improved error detection but is tailored to specific contexts like the German Federal Employment Agency's ALLEGRO system, lacking Spring Boot-specific analysis or seamless IDE integration [3]. Similarly, Bobade and Puri (2025) investigate ML techniques, including Random Forest and Support Vector Machines (SVM), for software defect prediction, emphasizing feature selection to improve accuracy [6]. However, their focus on predictive models rather than real-time quality assurance restricts applicability to immediate code improvement in Spring Boot environments [6].Khatami et al. (2025) introduce RepoInsights, a quality assurance analytics dashboard that provides insights into QA practices such as testing, code review, and automation [7]. While effective for fostering reflection among developers, the dashboard targets general metrics across open-source projects and does not address Spring Boot's layered architecture or framework-specific anti-patterns, such as misplaced business logic or inefficient transaction management [7].

Choudhury and Gupta (2025) explore ML integration into low-code/no-code (LC/NC) platforms for automated code review and error detection, demonstrating enhanced development speed and quality [5]. However, their focus on LC/NC environments limits applicability to traditional Spring Boot development, where explicit architectural patterns are critical [5]. Recent advancements in Spring Boot-specific AI applications provide a foundation for this project. Hamid (2024) discusses AI-driven test generation for Spring Boot using Java 23 and Spring Boot 3.3.4, leveraging large language models to automate unit testing [8]. CodeWithUs (2025) further explores AI-driven JUnit test generation, reducing manual testing efforts [9]. Memon (2024) introduces Spring AI for building smarter applications, suggesting potential for AI-driven quality assurance, though without focus on real-time IDE integration

[2]. These studies highlight AI's potential but are limited to test automation rather than comprehensive code quality analysis [2], [8], [9].

Additional research provides insights into AI-driven tools and IDE integration. Lenarduzzi et al. (2021) analyze the effectiveness of static analysis tools like SonarQube in detecting code smells, finding that only a small percentage of reported issues are relevant to developers, underscoring the need for context-aware tools [11]. Beller et al. (2016) investigate IDE-integrated code review tools, noting that real-time feedback significantly improves developer productivity but lacks framework-specific customization [12]. Kim and Lee (2019) propose an adaptive code recommendation system using ML to personalize suggestions based on developer behavior, though not tailored to Spring Boot or architectural patterns [13]. Fowler (2003) provides a theoretical foundation for architectural patterns like Layered and Hexagonal, which are central to Spring Boot's design and this project's architecture-aware analysis [10]. These studies collectively highlight the potential of AI and IDE integration but reveal gaps in framework-specific, real-time, and adaptive solutions.

Despite these advancements, significant limitations persist. Existing static analysis tools like SonarQube, Qodana, and Checkstyle rely on generic metrics that fail to account for Spring Boot's unique layer-based structure or architectural patterns, often missing anti-patterns like improper transaction boundaries or tight coupling [1] , [4] , [11]. ML-based approaches, while promising, are either too general or context-specific, lacking seamless IDE integration for real-time feedback [3, 6]. Moreover, current tools do not incorporate explicit architecture input or adapt to developer feedback, limiting alignment with project-specific conventions [5] , [7] , [13]. AI-driven test generation tools for Spring Boot show promise but do not address comprehensive code quality analysis [8] , [9]. This project addresses these gaps by developing an AI-driven IDE component that leverages a fine-tuned Code Llama model for real-time, architecture-aware analysis, computes dynamic layer-specific metrics, and adapts recommendations through developer feedback, specifically tailored for Spring Boot applications.

1.3. Research Gap

Despite advancements in software quality assurance (SQA) tools and machine learning (ML)-based code analysis, significant gaps remain in addressing the specific needs of Spring Boot development within integrated development environments (IDEs).

The literature reveals several critical gaps in software quality assurance (SQA) tools for Spring Boot development, particularly in the context of IDE-integrated, real-time, and adaptive code quality analysis. These gaps are:

Lack of spring boot-specific analysis:

Existing static analysis tools (e.g., SonarQube, Qodana, Checkstyle) rely on generic metrics like Lines of Code (LOC) and Cyclomatic Complexity (CC), failing to detect Spring Boot-specific anti-patterns such as business logic in Controllers, improper annotation usage (e.g., @Controller vs. @Service), or inefficient transaction management [1, 4, 11].

These tools do not account for Spring Boot's layered architecture (Controllers, Services, Repositories, Entities) or architectural patterns like Layered, Hexagonal, or Clean Architecture, leading to irrelevant recommendations [4, 10].

Limited real-time ide integration:

Current tools, such as those proposed by Kataieva and Nemec (2025) via Maven plugins, operate outside IDEs, requiring manual execution and lacking real-time feedback during coding [1].

Absence of architecture-aware metrics:

Tools do not incorporate explicit configuration of architectural patterns, relying on error-prone inference, which reduces the accuracy of quality metrics and recommendations [1, 10].

Metrics like Relative Complexity Index (RCI) and Interaction Complexity (IC) are not tailored to Spring Boot's layer-specific requirements (e.g., REST endpoint completeness, business logic cohesion) [1].

Lack of adaptive learning:

Existing tools, including ML-based approaches, do not adapt to team-specific coding conventions through developer feedback, limiting alignment with project standards [5, 7, 13].

Kim and Lee (2019) propose adaptive code recommendations, but their system is not tailored to Spring Boot or architectural patterns [13].

Narrow focus of ai-driven tools:

Recent advancements in AI applications for Spring Boot development, including test generation tools by Hamid (2024) and CodeWithUs (2025), demonstrate the potential of AI integration but remain narrowly focused on specific tasks rather than comprehensive code quality analysis [8, 9]. Memon (2024) suggests broader applications of AI in Spring Boot development but lacks concrete implementation for real-time quality assurance within development environments [2]. Machine learning approaches by Sultanow et al. (2025) and Bobade and Puri (2025) show promise in error detection and defect prediction but are either system-specific or focused on predictive modeling rather than real-time quality improvement [3, 6].

Explainability and actionability gap

Existing tools frequently provide quality metrics and issue identification without sufficient contextual explanation or actionable remediation guidance. The lack of explainable recommendations that reference specific Spring Boot best practices, architectural principles, or provide concrete code improvement suggestions limits the educational value and practical utility of quality assurance feedback, reducing developer adoption and long-term skill development.

This project addresses these gaps by developing an AI-driven IDE component that leverages a fine-tuned Code Llama model to provide real-time, architecture-aware analysis, compute dynamic layer-specific metrics, and adapt recommendations through developer feedback, specifically tailored for Spring Boot applications.

1.4. Research Problem

The rapid adoption of Spring Boot for enterprise microservices and web applications has introduced significant challenges in maintaining high code quality, necessitating a robust, framework-specific SQA solution. The research problem can be divided into the following key issues:

Prevalence of spring boot-specific anti-patterns:

Prevalence of Spring Boot-Specific Anti-Patterns: Issue: In Spring Boot development, developers often face recurring anti-patterns that undermine the framework's strengths in building microservices and web applications, such as embedding business logic directly into Controllers instead of Services, misusing annotations (e.g., applying @Controller where @Service is appropriate), and implementing inefficient transaction management without proper @Transactional boundaries [1, 2, 10]. These issues arise due to the framework's layered architecture—Controllers for request handling, Services for logic, Repositories for data access, and Entities for modeling—which, if violated, lead to tightly coupled code and poor separation of concerns, as highlighted in enterprise architecture patterns [10]. For instance, studies on code smells categorize such problems as bloaters or coupling issues, proposing metrics like Relative Complexity Index (RCI) to detect them, but these are not Spring Boot-specific [1].

Impact: Such anti-patterns significantly increase debugging time, as developers must manually trace errors across layers, reducing overall productivity—especially in complex projects following patterns like Layered, Hexagonal, or Clean Architecture [3]. This compromises maintainability, scalability, and reliability, leading to higher defect rates post-release, as empirical studies show that unaddressed code quality issues affect pull request acceptance and long-term system health [11].

Inadequacy of existing static analysis tools:

Issue: Traditional static analysis tools like SonarQube, Qodana, and Checkstyle depend on generic metrics such as Lines of Code (LOC) and Cyclomatic Complexity (CC), which overlook Spring Boot-specific nuances like improper dependency injection or framework-optimized query patterns, failing to provide architecture-aware feedback [1, 4, 11]. For example, while these tools can identify basic structural flaws, they do not account for Spring Boot's unique layered structure or integrate with advanced ML for hidden error detection in large-scale systems [3, 6]. Research on defect prediction using Random Forest and SVM

emphasizes feature selection for accuracy but focuses on general prediction rather than framework-tailored analysis [6].

Impact: The resulting irrelevant recommendations, misaligned with Spring Boot best practices, hinder effective code quality assurance, as only a fraction of reported issues prove actionable, reducing developer adoption and trust in these tools [4, 11].

Lack of real-time, ide-integrated feedback:

Issue: Many current tools, including Maven-based plugins for code metrics, operate externally to IDEs, requiring manual execution and lacking immediate feedback during coding sessions [1]. Even IDE-integrated solutions often miss Spring Boot-specific customizations, such as real-time layer validation or annotation checks, as seen in studies of modern code review practices [12]. This gap persists despite advancements in ML-based static analysis for quality assurance, which are not optimized for on-the-fly integration [3, 5].

Impact: Developers are forced to switch contexts to external dashboards or tools, increasing workflow friction, prolonging debugging cycles, and diminishing efficiency, particularly in fast-paced enterprise environments where rapid iterations are essential.

Absence of adaptive and personalized recommendations:

Issue: Existing tools do not adapt to team-specific coding conventions or incorporate developer feedback, limiting alignment with project standards [5, 7, 13].

Impact: Generic recommendations fail to address team-specific needs, reducing their practical utility and effectiveness.

Limited scope of ai-driven solutions:

Issue: AI-driven tools for Spring Boot focus on narrow tasks like test generation, not comprehensive quality analysis addressing layer-specific metrics or anti-patterns [8, 9].

Impact: The absence of holistic, real-time quality assurance limits the ability to improve overall code reliability and maintainability.

This project addresses these issues by developing an AI-driven code quality assurance engine integrated into IDEs, leveraging a fine-tuned Code Llama model to provide real-time, architecture-aware analysis, dynamic layer-specific metrics, and adaptive recommendations

tailored to Spring Boot's unique requirements, thereby enhancing code reliability, maintainability, and developer productivity.

Table 1: Comparison of Proposed Engine with Existing Tools and Systems

Tool/System	Spring Boot-Specific Analysis	Real-Time IDE Integration	Architecture-Aware Metrics	Adaptive Learning	AI-Driven Recommendations	Comprehensive Code Quality Analysis
Proposed Engine	✓	✓	✓	✓	✓	✓
SonarQube	◑	✓	✗	✗	◑	◑
Qodana	◑	✓	✗	✗	✗	◑
Checkstyle	✗	◑	✗	✗	✗	✗
DeepCode (Snyk Code)	✗	✓	✗	✗	✓	◑
RepoInsights	✗	✗	✗	✗	✗	◑
Spring AI-based Tools	◑	◑	✗	✗	◑	✗
Kataieva and Nemec (2025)	◑	✗	◑	✗	✗	◑
Sultanow et al. (2025)	✗	✗	✗	✗	✓	◑
Bobade and Puri (2025)	✗	✗	✗	✗	◑	✗
Choudhury and Gupta (2025)	✗	✗	✗	◑	✓	◑

✓ Full Support ◑ Partial Support ✗ No Support

2. OBJECTIVES

2.1. Main Objectives

The primary goal of this project is to develop an innovative AI-driven code quality assurance engine that enhances the development of Spring Boot applications by providing real-time, architecture-aware, and adaptive code quality analysis directly within popular IDEs. The main objectives are:

Enhance code quality in spring boot applications: Develop a tool that ensures adherence to Spring Boot best practices and architectural patterns, minimizing anti-patterns to improve maintainability, scalability, and reliability.

Boost developer productivity: Provide real-time, contextual feedback within the IDE to reduce debugging time, eliminate context-switching, and streamline the development process.

Enable architecture-aware analysis: Create a system that respects and leverages user-specified architectural patterns (e.g., Layered, Hexagonal, Clean) to deliver precise, relevant quality metrics and recommendations.

Promote adaptive and personalized assistance: Implement an adaptive learning mechanism that tailors recommendations to team-specific coding conventions, enhancing tool effectiveness and developer adoption.

2.2. Specific Objectives

To achieve the main objectives, the project will focus on the following specific, measurable objectives:

Develop an ide-integrated plugin:

Implement a plugin compatible with IntelliJ IDEA, enabling seamless integration into existing development workflows.

Ensure the plugin supports real-time analysis triggered by code changes (e.g., on-save or on-the-fly typing) to provide immediate feedback.

Enable architecture configuration:

Design a setup wizard or configuration file (e.g., YAML/JSON) for developers to specify the project's architectural pattern (Layered, Hexagonal, or Clean) during initialization.

Achieve 100% accuracy in layer classification (e.g., identifying Controllers, Services, Repositories, Entities) based on annotations, package structure, and configured architecture.

Compute layer-specific quality metrics:

Develop algorithms to compute dynamic metrics for each Spring Boot layer, including:

Controller Layer: REST endpoint completeness (e.g., proper annotations, input validation), response consistency (e.g., use of ResponseEntity).

Service Layer: Business logic cohesion, transaction management correctness (e.g., proper use of @Transactional).

Repository Layer: Query efficiency (e.g., avoiding N+1 select issues), adherence to Spring Data JPA standards.

Entity Layer: Data model integrity (e.g., correct use of @Entity, validation annotations).

Output numerical scores (0–100) or categorical ratings (High/Medium/Low) displayed in the IDE with explanations.

Implement AI-Driven Recommendations:

Fine-tune a Code Llama model on datasets of Spring Boot code, best practices, and anti-patterns from GitHub repositories and empirical studies.

Generate contextual, explainable recommendations with:

- Rationale referencing Spring Boot best practices or architectural principles (e.g., SOLID, Clean Architecture).
- Links to resources (e.g., Spring documentation, Stack Overflow).
- Actionable code snippets with one-click apply options in the IDE.

Achieve at least 90% accuracy in detecting Spring Boot-specific anti-patterns (e.g., business logic in Controllers, improper transaction boundaries).

Incorporate Adaptive Learning:

Develop a feedback loop to track developer interactions (e.g., accepting, ignoring, or modifying suggestions) and fine-tune the AI model locally or via a privacy-compliant cloud backend.

Enable the plugin to adapt recommendations to team-specific conventions within 50–100 feedback cycles, reducing irrelevant suggestions by at least 80%.

3. METHODOLOGY

3.1. System Architecture

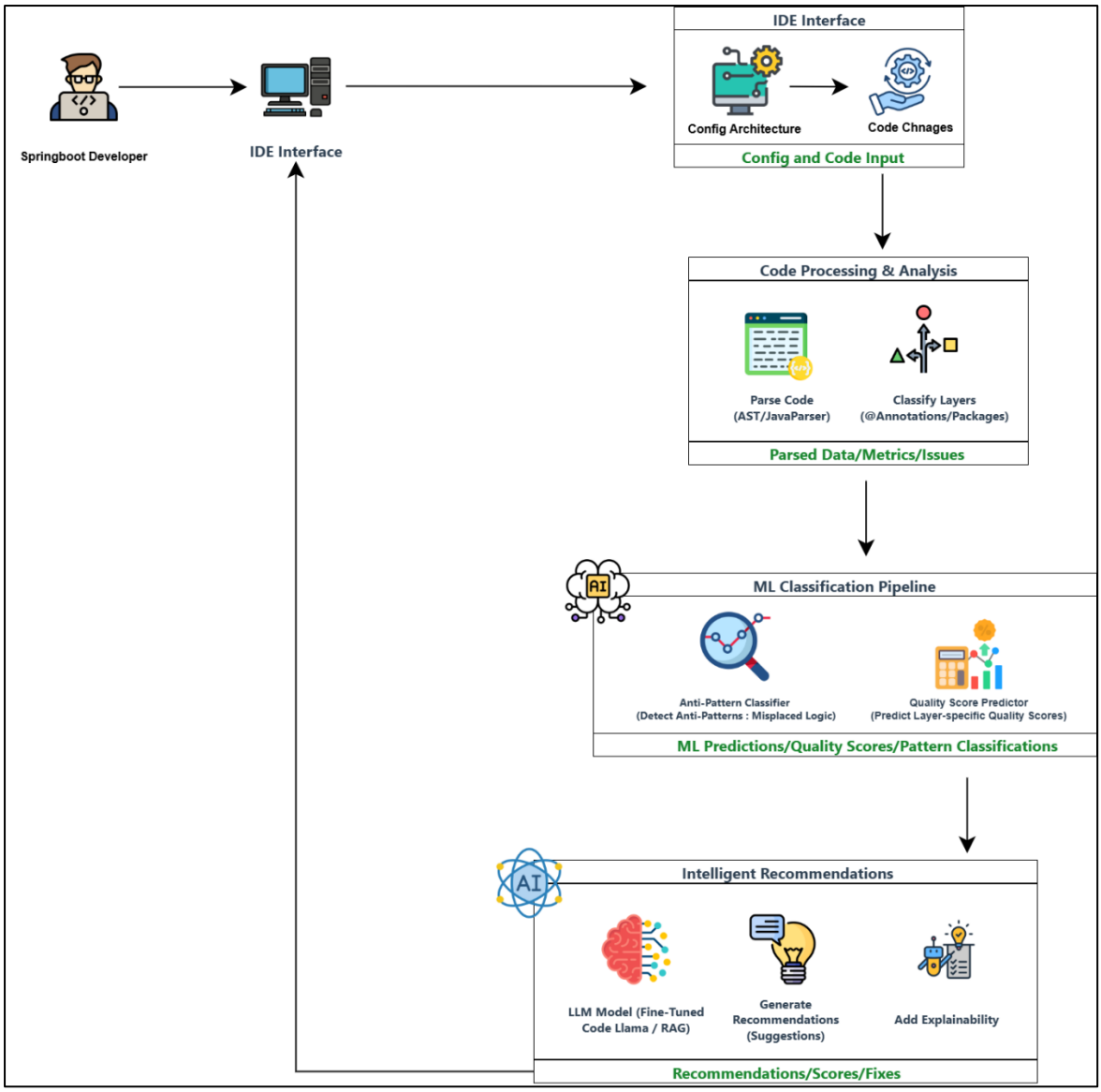


Figure 2 : System Architecture Diagram

The system architecture for the AI-Driven Code Quality Spring Boot Assurance Engine is designed to seamlessly integrate into popular Integrated Development Environments (IDEs) IntelliJ IDEA, providing real-time, architecture-aware code quality analysis tailored to Spring Boot applications. The architecture operates through a multi-tiered approach, beginning with the User Tier where developers configure the project's architectural pattern (e.g., Layered, Hexagonal, or Clean) and edit code, triggering real-time analysis. The Analysis Tier processes this input by parsing code with Abstract Syntax Tree (AST) tools, classifying layers based on

annotations and package structures, computing dynamic quality metrics (e.g., cohesion, query efficiency), and detecting anti-patterns specific to Spring Boot. The AI Tier leverages a fine-tuned Code Llama model to generate contextual recommendations, enhance explainability with references to best practices, and provide actionable fixes. Feedback from developers is captured and processed in the Adaptation Tier, enabling continuous model refinement and personalization to align with team-specific conventions. This closed-loop system ensures scalability, handling projects up to 100,000 lines of code and supporting 50 concurrent users, while maintaining performance (1-2 seconds per analysis) and reliability across long coding sessions. The architecture addresses the identified research gaps by offering a robust, adaptive solution that enhances code reliability, maintainability, and developer productivity in Spring Boot development.

3.2. Software Solution

The software solution for the AI-Driven Code Quality Spring Boot Assurance Engine is a comprehensive approach to designing, developing, and deploying an IDE-integrated plugin that enhances code quality in Spring Boot applications. This solution encompasses requirement gathering, feasibility studies (economic, scheduled, and technical), and a structured development process to ensure the plugin meets the project's objectives.

Requirement Gathering

Requirements for the AI-Driven Code Quality Spring Boot Assurance Engine were established by examining the obstacles encountered by Spring Boot developers, including maintaining architectural integrity, identifying framework-specific anti-patterns, and obtaining tailored feedback. These requirements were divided into functional needs, such as on-the-fly code analysis triggered by edits, layer identification using annotations and configured architecture, calculation of dynamic quality metrics (e.g., cohesion, query performance), detection of anti-patterns, and delivery of contextual suggestions with executable fixes via a fine-tuned Code Llama model. Non-functional requirements encompass a response time of 1-2 seconds for analysis, 90% accuracy in detecting anti-patterns, smooth integration with IDEs like IntelliJ IDEA, Eclipse, and VS Code, and the capacity to manage projects up to 100,000 lines with 50 simultaneous users. Insights were gathered from existing research on software quality assurance, Spring Boot guidelines, and machine learning applications in code review, refining the plugin’s focus to bridge gaps in tools such as SonarQube and Qodana.

Feasibility Study

Economic Feasibility – The proposed solution proves cost-effective by utilizing open-source tools like JavaParser for AST parsing, pre-trained Code Llama models, and local computation to keep infrastructure expenses low. Initial costs, estimated at LKR 15,000 (~USD 50), include internet access, online courses for LLM optimization, and materials for presentations/demos, which can be recuperated through a potential premium plugin subscription on platforms like JetBrains Marketplace. The anticipated benefits—decreased debugging efforts, enhanced code stability, and boosted developer efficiency—significantly surpass the upfront costs, ensuring economic viability for both individual users and enterprises.

Scheduled Feasibility – The development plan spans 4 months (August-November 2025), synchronized with the academic timeline, and is organized into iterative sprints. Key stages comprise requirement analysis and design (1 month), coding and integration (2 months), and testing and assessment (1 month). This sprint methodology guarantees the timely rollout of essential features—such as layer detection, metric evaluation, and AI-powered suggestions—while offering flexibility to tackle issues, minimizing the likelihood of missing deadlines.

Technical Feasibility – The system is technically viable, incorporating well-established technologies such as Java 17 and Spring Boot 3.0+ for compatibility, AST utilities (e.g., JavaParser) for code breakdown, and a fine-tuned Code Llama model for AI insights, all backed by Texlive-full packages. The plugin is built using IntelliJ IDEA development frameworks (Java/Kotlin), with the option for local or secure cloud processing to ensure scalability. The developer's solid foundation in Spring Boot and machine learning, augmented by specialized online training, alongside minimum hardware needs (8GB RAM, 2GHz CPU), supports feasibility. Challenges like optimizing real-time performance are addressable through pre-trained models and streamlined algorithms.

3.3. Commercialization

The AI-Driven Code Quality Spring Boot Assurance Engine can be commercialized as a premium IDE plugin for IntelliJ IDEA, Eclipse, and VS Code, targeting enterprise Java developers and teams building Spring Boot applications. Offered via a subscription model (e.g., monthly/yearly licenses) on platforms like JetBrains Marketplace, it ensures recurring revenue. Additional revenue streams include professional services for custom integrations and training for corporate teams. Marketing will emphasize its architecture-aware analysis, real-time feedback, and adaptive learning, appealing to startups and large organizations aiming to enhance code quality and developer productivity. Partnerships with Spring Framework

communities and cloud providers can boost adoption, aligning with industry demand for AI-driven development tools.

3.4. Future Scope

Future enhancements include supporting additional Java frameworks (e.g., Quarkus, Micronaut) and integrating with cloud-based IDEs like GitHub Codespaces. Expanding the AI model to cover emerging Spring Boot versions and incorporating multi-language support (e.g., Kotlin) will broaden applicability. Adding predictive analytics for code maintainability and team collaboration features can further enhance functionality. Open-sourcing parts of the plugin could foster community contributions, driving innovation and adoption in the developer ecosystem.

4. PROJECT REQUIREMENTS

4.1. Functional Requirements

The AI-Driven Code Quality Spring Boot Assurance Engine must provide the following functionalities to meet the project's objectives and address the identified research gaps:

IDE Integration:

The component must function as a plugin compatible with IntelliJ IDEA

It must provide real-time code quality analysis triggered by code changes, such as saving a file or typing, to deliver immediate feedback within the IDE.

Architecture Configuration:

The component must include a setup wizard or configuration file (e.g., YAML or JSON) allowing developers to specify the project's architectural pattern (Layered, Hexagonal, or Clean) during initialization.

It must store and validate the architecture configuration to ensure recommendations align with the specified pattern.

Layer Identification:

The component must accurately identify Spring Boot architectural layers (Controllers, Services, Repositories, Entities) based on annotations (e.g., @Controller, @Service), package structure, and the configured architecture.

It must achieve 100% accuracy in layer classification to ensure reliable metric computation and issue detection.

Layer-Specific Quality Metrics:

The component must compute and display dynamic quality metrics for each Spring Boot layer.

Metrics must be presented as numerical scores (0–100) or categorical ratings (High/Medium/Low) in the IDE, accompanied by explanations.

Anti-Pattern Detection:

The component must detect Spring Boot-specific anti-patterns, such as business logic in Controllers, improper annotation usage, inefficient transaction management, or tight coupling between layers.

It must achieve at least 90% accuracy in identifying these anti-patterns, validated through user testing.

AI-Driven Recommendations:

The component must leverage a fine-tuned Code Llama model, trained on datasets of Spring Boot code, best practices, and anti-patterns from GitHub repositories and empirical studies.

It must generate contextual recommendations, including:

Rationale referencing Spring Boot best practices or architectural principles (e.g., SOLID principles, Clean Architecture).

Links to authoritative resources (e.g., Spring documentation, Stack Overflow).

Adaptive Learning:

The component must implement a feedback loop to track developer interactions (e.g., accepting, ignoring, or modifying suggestions).

It must use anonymized feedback to fine-tune the AI model locally or via a privacy-compliant cloud backend, adapting recommendations to team-specific conventions within 50–100 feedback cycles.

Feedback Visualization:

The component must display metrics, detected issues, and recommendations in the IDE.

4.2. Non-Functional Requirements

The component must meet the following non-functional requirements to ensure usability, performance, and reliability:

Performance:

Real-time analysis must complete within 1 second for typical code changes (e.g., editing a single file with up to 1,000 lines) to ensure a seamless developer experience.

The component must process projects with up to 100,000 lines of code without exceeding 2 seconds for full analysis.

Accuracy:

The component must achieve at least 90% accuracy in detecting Spring Boot-specific anti-patterns and classifying architectural layers, validated through user testing on real-world projects.

Recommendations must have ≥85% developer agreement on relevance, as measured by user feedback during evaluation.

Usability:

Recommendations and metric explanations must be clear and understandable to developers with intermediate Spring Boot knowledge (e.g., 1–2 years of experience).

The setup wizard must be intuitive, requiring no more than 5 minutes to configure the architectural pattern for a typical project.

Scalability:

The component must handle Spring Boot projects with up to 100,000 lines of code and 50 concurrent users (e.g., in a team setting) without significant performance degradation.

The AI model must scale to process feedback from multiple developers without requiring excessive computational resources.

Reliability:

The component must operate without crashes or interruptions during continuous IDE usage (e.g., 8-hour coding sessions).

It must maintain consistent performance across supported IDEs and operating systems.

Privacy:

Adaptive learning must anonymize developer feedback data to comply with data protection regulations (e.g., GDPR).

The component must support local processing of the AI model to avoid external data transmission unless explicitly authorized by the user.

4.3. System Requirements

The component must meet the following system requirements to ensure compatibility and deployment feasibility:

- **Hardware**:
  - Minimum: 8GB RAM, 2GHz quad-core CPU, 500MB disk space for plugin installation and AI model processing.
  - Recommended: 16GB RAM, 3GHz quad-core CPU, 1GB disk space for optimal performance on large projects.
- **Software**:
  - Compatible with IntelliJ IDEA (2023.1 or later), Eclipse (2023-06 or later), and VS Code (1.80 or later).
  - Requires Java 17 or later and Spring Boot 3.0 or later for project analysis.
  - Depends on libraries such as JavaParser or Eclipse JDT for Abstract Syntax Tree (AST) parsing and a fine-tuned Code Llama model for AI-driven analysis.
- **Operating Systems**:
  - Supported on Windows 10 or later, macOS 12 or later, and Linux (Ubuntu 20.04 or later) to ensure broad accessibility.
- **Network** (Optional):
  - For cloud-based adaptive learning (if enabled), a stable internet connection with at least 10 Mbps bandwidth is required.
  - Local processing must be supported to ensure functionality in offline environments.

4.4. Work breakdown structure

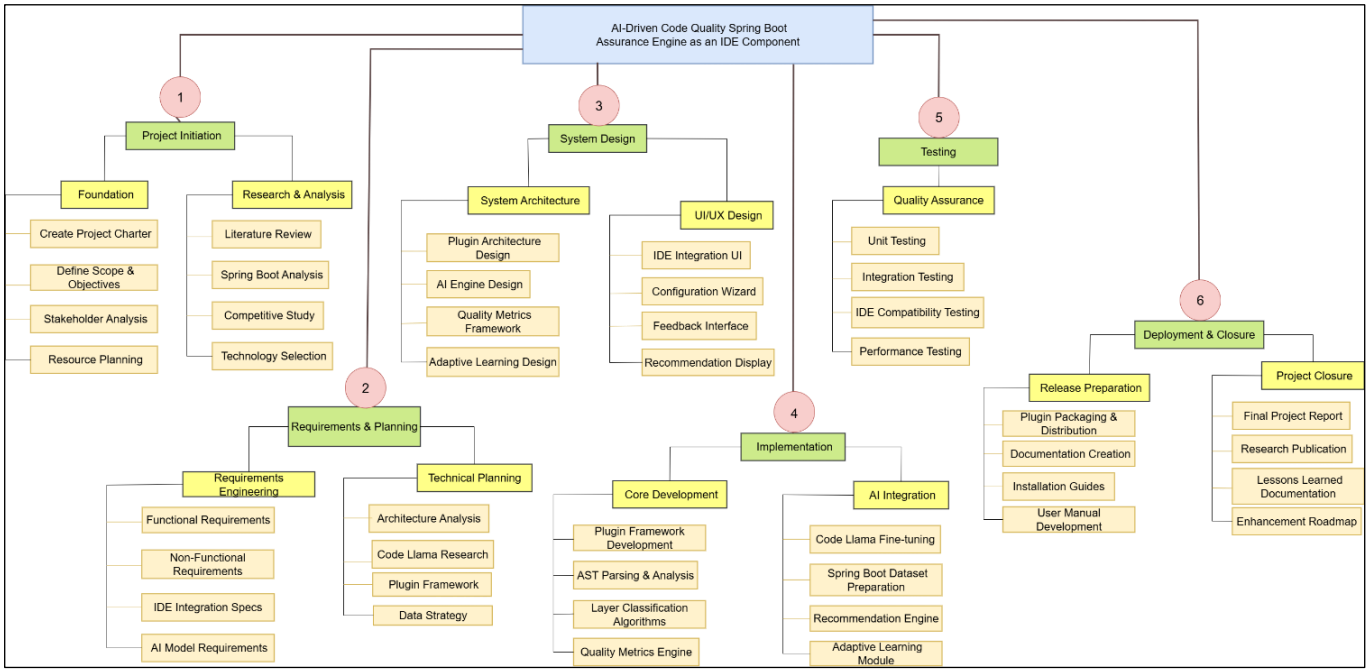
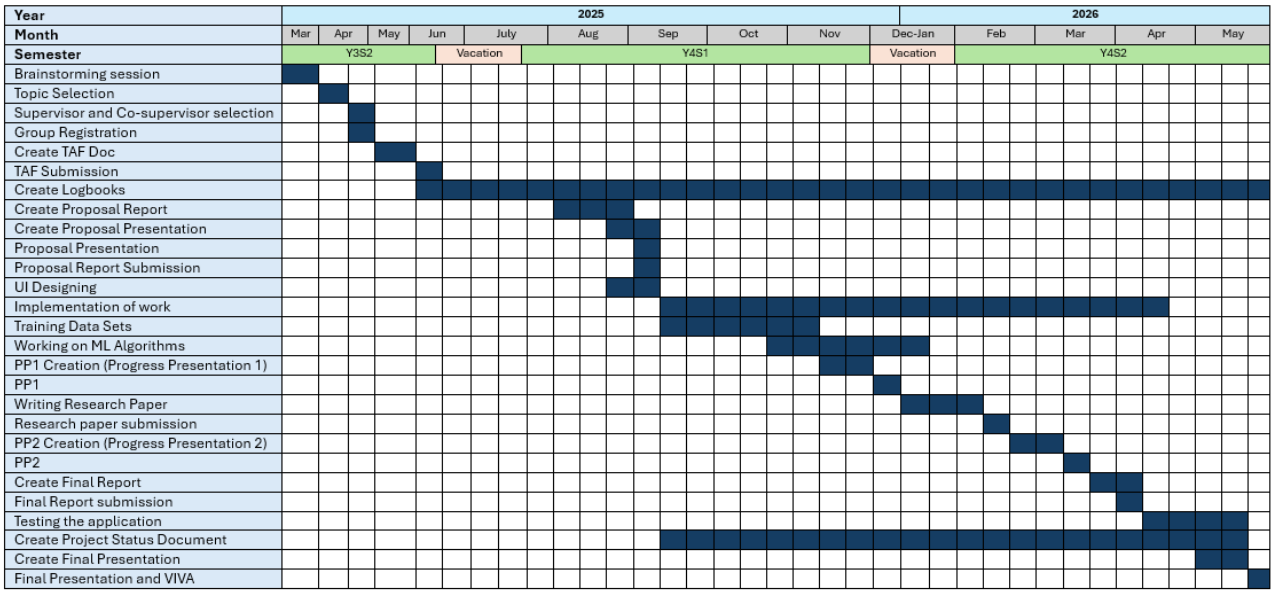


Figure 3 : Work breakdown structure

4.5. Gantt Chart

Table 2 : Gantt Chart



5. BUDGET JUSTIFICATION

Item	Cost (LKR)	Cost (USD)	Description
Internet Charges	6,000	20	Covers mobile data plans for accessing technical documentation, open-source repositories, and online tutorials during research, development, and testing phases.
Technical Charges (Online Courses)	3,000	10	Covers a single online course on platforms like Udemy or Coursera to learn fine-tuning LLMs with QLoRA, IntelliJ plugin development, or Spring AI integration.
Miscellaneous (Software, Presentation, Demo)	6,000	20	Covers presentation materials (e.g., A2 poster for presentation), domain for demo website, and minor hosting costs.
Total	**15,000**	**50**	

Notes

- **Currency Conversion**: Costs are calculated at 1 USD ≈ 300 LKR, reflecting August 2025 exchange rates in Sri Lanka.

6. REFERENCES

[1] Kataieva, Y., & Nemec, M. (2025). Determining software quality using code analysis metrics. *29th International Conference on Information Technology (IT)*, ˇZabljak, 19–22 February, 2025.

[2] Memon, F. (2024). Build smarter Spring Boot applications with Spring AI. *freeCodeCamp*. https://www.freecodecamp.org/news/build-smarter-spring-boot-applications-with-spring-ai/

[3] Sultanow, E., Konopik, S., Ullrich, A., & Vladova, G. (2025). Machine learning based static code analysis for software quality assurance. *Capgemini and Dept. of Business Informatics, University of Potsdam*. https://www.lswi.de/fileadmin/user_upload/lswi/publications/2025/Sultanow_Konopik_Ullrich_Vladova_2025.pdf

[4] Khatami, A., Brandt, C., & Zaidman, A. (2025). Software quality assurance analytics: Enabling software engineers to reflect on QA practices. *Delft University of Technology*.

[5] Choudhury, D., & Gupta, D. (2025). Investigation on integration of machine learning techniques into LC/NC platforms for code review, quality assessment, and error detection automation. *Amity Institute of Information Technology, Amity University*.

[6] Bobade, V., & Puri, C. (2025). Improving software defects detection: An in-depth analysis of machine learning methods and static analysis tools for greater accuracy. *Department of Artificial Intelligence and Machine Learning, Faculty of Engineering And Technology, DMIHER*.

[7] Salm, T. (2025). Building AI-driven applications with Spring AI. *JAVAPRO*. https://javapro.io/2025/04/22/building-ai-driven-applications-with-spring-ai/

[8] Hamid, A. K. (2024). Java 23, SpringBoot 3.3.4: AI-driven test generation. *Medium*. https://medium.com/illumination/java-23-springboot-3-3-4-ai-driven-test-generation-6fd2cb12d153

[9] CodeWithUs. (2025). You'll never write another JUnit test again—Thanks to AI in Spring Boot. *Medium*. https://medium.com/@kanhaaggarwal/youll-never-write-another-junit-test-again-thanks-to-ai-in-spring-boot-dba89afc3907

[10] Fowler, M. (2003). Patterns of enterprise application architecture. *Addison-Wesley*. https://www.martinfowler.com/books/eaa.html

[11] Lenarduzzi, V., Lomio, F., Saarimäki, N., & Taibi, D. (2021). Does code quality affect pull request acceptance? An empirical study. *Journal of Systems and Software*, 171, 110806. https://doi.org/10.1016/j.jss.2020.110806

[12] Beller, M., Bacchelli, A., Zaidman, A., & Juergens, E. (2016). Modern code review: A case study at Google. *2016 IEEE/ACM 38th International Conference on Software Engineering Companion (ICSE-C)*, 181–190. https://doi.org/10.1145/2889160.2889243

[13] Kim, S., & Lee, J. (2019). Adaptive code completion using machine learning. *2019 IEEE/ACM 41st International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*, 61–64. https://doi.org/10.1109/ICSE-NIER.2019.00024

APPENDICES